

Progetto WebApp

BOARD GAMES LIBRARY

MICHELA PAIARIN - 163250

SOMMARIO

BoardGamesLibrary è un'applicazione web full-stack per la catalogazione, l'organizzazione e il filtraggio di una collezione privata di giochi da tavolo, realizzata seguendo il paradigma REST. Per la realizzazione sono state adottate diverse tecnologie quali: **NODE.JS** e **EXPRESS.JS** per l'ambiente server insieme a **SQLITE** per la gestione dei dati. **HTML5**, **JAVASCRIPT** e **TAILWIND CSS V4** per quanto riguarda, invece, lo sviluppo dell'interfaccia utente a livello, rispettivamente, strutturale, funzionale e grafico.

L'obiettivo è stato quello di realizzare un'applicazione che non richiedesse framework complessi permettendo la creazione di un sistema corretto, sicuro e manutenibile. Il focus, quindi, è stato posto sul mantenere la separazione rigorosa delle responsabilità tra i moduli creati in modo da rendere l'applicazione scalabile e di facile comprensione. Un altro punto cruciale è la sicurezza dei dati sia rispetto a SQL Injection sia rispetto alla consistenza logica degli stessi. A supporto di questi obiettivi, i validatori sono stati implementati come moduli condivisi tra i due livelli applicativi, garantendo un'unica fonte di verità sui vincoli accettati dal sistema.

Sul fronte del front-end, si è scelto di realizzare una Single Page Application dinamica gestita interamente da JavaScript, senza l'utilizzo di framework esterni.

MODELLO DEI DATI

L'entità centrale del progetto è **GAMES** che astrae i dati principali di un classico gioco da tavolo. Ogni record è identificato da un ID autoincrementale che lo identifica univocamente all'interno della collezione e da un nome univoco, a cui si affiancano i campi obbligatori: *MinPlayer*, *MaxPlayer*, *MinAge*, *Time* e *Location*. Tali campi vanno a memorizzare, rispettivamente, i valori legati al numero di giocatori, l'età minima, il tempo di gioco stimato e la sua collocazione all'interno delle mura domestiche in formato *stanza.armadio.ripiano*. Sono, invece, facoltativi: *Description*, *Year*, *UrlBigImage* e *UrlSmallImage* che memorizzano descrizione, anno di pubblicazione e url per eventuali immagini di copertina.

Il validatore svolge, quindi, un ruolo fondamentale in questa fase: verifica la correttezza dei tipi primitivi, i formati stringa ammessi (es. *Location* tramite regex) e i vincoli logici derivati dalla natura dei dati (es. *MinPlayer* $\leq$ *MaxPlayer*, *Year* $\leq$ *anno corrente*, *Time* $>$ 0). Prima che i dati raggiungano il validatore è necessario un passaggio di pulizia dei dati. Tale passaggio è gestito dal controller e volto a eliminare le stringhe vuote, i valori nulli e convertire le stringhe numeriche in numeri. Dopo la validazione, il controller verifica anche l'unicità del nome tramite query al database.

Tale schema è una semplificazione consapevole di un'architettura logica più articolata riportata come schema nella cartella **./doc** della repository. Tale schema prevederebbe entità separate per stanza, armadio e ripiano e l'inserimento di ulteriori entità per caratterizzare maggiormente i dati memorizzati quali i generi e le espansioni. Questa scelta è stata fatta per garantire stabilità al sistema e rispettare i tempi di consegna, mantenendo aperta la possibilità di estensione futura.

MODULI CONDIVISI

Una scelta centrale del progetto è l'adozione di un livello condiviso tra back-end e front-end che si trova memorizzato nella cartella **/shared**. Questo strato ospita tutti i dati e i moduli che sono comuni a entrambi i livelli quali i validatori, i file di configurazione dei vincoli e i dizionari dei dati statici dell'applicazione. Il vantaggio di questa scelta è garantire che back-end e front-end operino sempre con le stesse regole senza che queste vengano ripetute in entrambe le strutture andando incontro a rischi di inconsistenza. Una modifica, per esempio, al vincolo minimo dell'età si propaga direttamente a entrambi i livelli senza rischio di divergenza eliminando bug quali accettazione di dati da parte del front-end rifiutati poi però dal back-end. Inoltre, permette di non ripetere porzioni logiche analoghe a entrambe le parti quali i metodi di validazione dei dati.

Il modulo **sharedExports** presente da entrambi i lati funge da barrel file centralizzando tutti i moduli condivisi offrendo un unico punto di importazione sia al back-end che al front-end. In questo modo ogni livello accede allo stesso codice con path diversi e senza duplicazioni.

I **validators** implementano la validazione isomorfa dei dati passati esponendo metodi di pulizia, normalizzazione e controllo dei vincoli. Essi rappresentano moduli concreti di quella che è la classe astratta **common**. Tale modulo, infatti, racchiude i metodi comuni a tutti i validatori quali la pulizia dei dati o i controlli di tipo. Sono poi i validatori specifici, quello per l'entità *games* e quello per i filtri, a conoscere i vincoli che questi dati devono rispettare. Essi, infatti, hanno il compito di prendere i parametri di validazione, tradurli e chiamare i metodi corretti del modulo comune per verificarli. Tali parametri sono centralizzati in specifici file di configurazione presenti nella cartella */constraints* e condivisi in modo da garantire coerenza e facilitare future modifiche. Essi non contengono nessuna parte logica in quanto hanno l'unico compito di memorizzare le informazioni necessarie.

BACK-END

ARCHITETTURA

Il server Express adotta un'architettura a strati con separazione netta delle responsabilità. Di seguito si analizza ogni modulo seguendo il percorso di una richiesta.

Il **server** inizializza Express stabilendo la connessione al database e delegando la gestione delle rotte al router di competenza. La struttura è pensata per essere scalabile: aggiungere una nuova entità oltre a */games* richiederebbe la creazione di una copia dei vari moduli elencati di seguito senza modificare quelli esistenti.

Il **router** riceve le richieste HTTP svolgendo un ruolo da intermediario fra il protocollo utilizzato per le richieste e la logica dell'applicativo. Esso, infatti, traduce le richieste HTTP in chiamate ai metodi del controller, trasformando i parametri di percorso e il body in variabili JavaScript leggibili dal livello applicativo e le risposte in JSON. La gestione degli errori è centralizzata nel router tramite la funzione *handleError*, che mappa le eccezioni strutturate lanciate dal controller in risposte HTTP con il codice di stato e i dettagli appropriati, garantendo un formato di risposta uniforme per tutti gli endpoint.

Il **controller** contiene la logica di business: riceve i dati dal router, li valida e li pulisce tramite il validator, descritto nella sezione precedente, verifica i vincoli di unicità e li inoltra al model. Esso, quindi, garantisce l'integrità e la purezza dei dati segnalandone eventuali errori di forma (400, *Bad Request*), di conflitti (409, *Conflicted*) o di riferimenti (404, *Not Found*). Ogni operazione è protetta da blocchi *try/catch* con log strutturati che prevengono crash imprevisti del server.

Il **model** è l'unico componente applicativo autorizzato a dialogare direttamente con i dati, essendo, assieme allo script di inizializzazione del database, l'unico a conoscenza del metodo di memorizzazione utilizzato. Traduce le richieste ricevute dal controller in query SQL parametrizzate con il supporto del **query builder**. Quest'ultimo converte oggetti JavaScript complessi in clausole WHERE dinamiche e sicure, azzerando strutturalmente il rischio di SQL Injection.

ROTTTE

Le rotte implementate coprono le operazioni CRUD standard sull'entità Games:

Metodo	Percorso	Descrizione
GET	/games	Restituisce la lista dei giochi (con filtri)
GET	/games/:id	Restituisce un gioco specifico
POST	/games	Crea un nuovo gioco
PUT	/games/:id	Aggiorna un gioco esistente
DELETE	/games/:id	Elimina un gioco

I codici di stato HTTP rispettano la semantica del protocollo: 200 *OK*, 201 *Created*, 400 *Bad Request*, 404 *Not Found*, 409 *Conflict*.

L'endpoint GET */games* supporta un sistema di filtraggio dinamico: i parametri di query vengono validati (correttezza, validità e conflitti) dal validator e poi tradotti dal query builder in operatori SQL di confronto (=, >, <, >=, <=, *LIKE*). Tale scelta permette di generare interrogazioni flessibili e arbitrarie direttamente dalle stringhe di query. Il sistema gestisce anche parametri virtuali come *Player* tradotto in un range su *MinPlayer* e *MaxPlayer* e *Room* che filtra per prefisso sul campo *Location*. Le richieste logicamente impossibili (es. *MinPlayer_ge* = 3 combinato con *MinPlayer_eq* = 5) vengono rifiutate con 400 *Bad Request* mentre le query valide che non producono risultati restituiscono 200 *Ok* con array vuoto.

ARCHITETTURA

Il front-end della pagina è una Single Page Application modulare in JavaScript.

Il **main** gestisce il ciclo di vita della pagina. Esso intercetta gli eventi di navigazione e gestisce il routing lato client senza mai ricaricare. Tale meccanismo è realizzato creando una pagina **index** contenente solo lo scheletro strutturale; tutti i componenti e le viste vengono caricati e sostituiti dinamicamente al suo interno.

Il **loader** è il delegato al caricamento ricevendo l'ID del contenitore di destinazione e conoscendo il percorso del componente specifico richiesto. Ogni vista dispone di un codice HTML per la struttura, un modulo JavaScript per la logica di popolamento e la gestione del DOM e un file CSS per gli stili specifici.

Tra i moduli di utilità condivisi da tutte le viste vi sono **api** che centralizza l'interazione con l'infrastruttura server, **notifier** che si occupa di un sistema di messaggistica visiva popup per l'esibizione di messaggi specifici all'utente e **filterManager** che si occupa della gestione dei filtri.

Il modulo **api** centralizza le chiamate *HTTP* al server richiedendo i dati necessari al front-end. Tramite un rigoroso controllo *try/catch* sui blocchi *fetch* e l'applicazione del pattern fail-fast: qualsiasi risposta con codice non 2xx viene immediatamente convertita in un'eccezione strutturata **ApiError**, forzando l'ingresso nel blocco *catch* anche per errori applicativi come 409 o 404.

La gestione visiva degli errori *HTTP* è delegata al modulo **notifier**, che li traduce in messaggi popup basati sull'elemento nativo `< dialog >` di HTML5, stilizzati in coerenza con la palette dell'applicazione sostituendosi alle funzioni native di sistema.

La gestione dei filtri attivi è centralizzata nel modulo **filterManager**, che mantiene un oggetto di stato globale aggiornato da tutte le componenti che interagiscono con la ricerca. Il modulo espone metodi per aggiungere e rimuovere filtri e notifica le viste dell'aggiornamento, evitando che ciascuna componente gestisca autonomamente una propria copia dello stato.

Due scelte implementative meritano particolare attenzione. In primo luogo, il calcolo delle statistiche aggregate (totale giochi, ore di gioco complessive, medie di giocatori, età e durata) viene eseguito direttamente nel browser tramite funzioni native sui vettori JavaScript azzerando l'esigenza di query aggregate sul database. Questo permette l'aggiornamento in tempo reale delle statistiche al variare dei filtri attivi operando sempre sul sottoinsieme di giochi attualmente visualizzato.

Analogamente, la gestione degli elementi del DOM e dei percorsi dei file da caricare è affidata a dizionari di configurazione centralizzati, seguendo il pattern Configuration-Driven Programming. Questo riduce i blocchi condizionali, garantisce un unico punto di aggiornamento in caso di modifiche strutturali e rende il codice più leggibile.

UI/UX

Per gli stili è stato adottato Tailwind CSS v4 come framework grafico esterno. L'approccio scelto, però, è diverso da quello standard Utility-First puro del sistema che prevede di aggiungere le classi CSS direttamente nel markup HTML. La scelta, infatti, è stata per un approccio semantico ibrido: il markup HTML contiene solo classi e ID semantici, mentre l'aspetto visivo è centralizzato nei file CSS attraverso la direttiva *@apply*. Questo mantiene il codice HTML leggibile e incentrato solamente sulle informazioni strutturali e concentra tutta la logica grafica nei file dedicati. Tale scelta rende più semplici eventuali modifiche future agli stili senza dover toccare il markup. Se, per esempio, decidessi di cambiare effetto transitorio a tutti i bottoni mi basterebbe modificare le classi *@apply* del tag `button` nel file CSS specifico. La strategia grafica adottata prevede un tema globale con variabili CSS custom in *@theme*, stili semantici per i componenti riutilizzati (*gameCard*, *popup*, *button*) e stili di vista specifici separati.

Il layout è fluido e responsive alternando proprietà Flexbox dinamiche e media query dove necessario, in modo che la pagina risulti navigabile anche su schermi di dimensioni ridotte.

Particolare attenzione, inoltre, è stata dedicata alla gestione degli stati limite dell'interfaccia. Quando la libreria è vuota o i filtri non producono risultati, vengono caricati dinamicamente componenti HTML dedicati con call-to-action contestuali, evitando schermate bianche prive di indicazioni. Analogamente, il form di modifica rileva se l'utente ha apportato modifiche non salvate e, in caso di navigazione, richiede conferma prima di abbandonare la pagina. Per concludere, i filtri rapidi gestiscono lato client le incompatibilità logiche tra di essi: attivare "*Solitario*", per esempio, disattiva automaticamente "*Famiglia*" e "*Party*", mantenendo lo stato della UI sempre consistente con i dati che verranno richiesti al server senza richiedere all'utente di prestare attenzione a queste incompatibilità. Questo impedisce che vengano inviate al server richieste logicamente impossibili, che verrebbero comunque bloccate dal validatore con un errore 400.

CONCLUSIONI

Il progetto prevede un sistema CRUD completo con validazione robusta, un motore di ricerca dinamico e un'interfaccia SPA funzionale. La scelta di non adottare framework reattivi per il front-end ha imposto una gestione esplicita dello stato e del DOM, ma ha permesso di comprendere e controllare ogni aspetto del ciclo di vita della pagina.

Le scelte architetturali del progetto convergono verso un obiettivo comune: minimizzare il costo del cambiamento e permettere una gestione del codice più chiara e facile durante la fase di realizzazione e di mantenimento. Il rispetto del principio di singola responsabilità prevede che ogni modulo abbia un solo compito specifico garantendo che le modifiche non abbiano effetti a cascata. Essi, infatti, sono specializzati e mirati assicurando ottimizzazione e chiarezza dal punto di vista strutturale e logico. Inoltre, rende l'applicativo suddivisibile in porzioni più piccole e scalabili e, quindi, di più facile realizzazione rispetto a strutture complesse. L'adozione di dizionari di configurazione centralizzati riduce i punti in cui un'informazione deve essere aggiornata oltre a eliminare innumerevoli blocchi *if/else*. La condivisione dei validatori tra i livelli elimina la possibilità di divergenza tra le regole del client e quelle del server. L'eliminazione delle ridondanze data dall'estrapolazione di logiche ripetute in metodi o moduli specifici, esempio *common*, rende il codice di facile modifica e mantenimento. Ogni scelta, presa singolarmente, è una buona pratica; prese insieme, costruiscono un sistema in cui aggiungere una funzionalità o correggere un vincolo richiede di toccare il minor numero possibile di file.

Il sistema, inoltre, è strutturato per essere scalabile permettendo di aggiungere ulteriori viste, per quanto riguarda il front-end, oppure ulteriori entità, per quanto riguarda il back-end. Dal punto di vista architetturale, il sistema è già predisposto per entrambe le modifiche senza richiedere cambiamenti ai moduli esistenti.